Course 2311: Analysis of Algorithms

Afsana Tasnim
Lecturer
Department of CSE
North Bengal International University

Optimization Problems

- Optimization Problem
  - Problem with an objective function to either
    - Maximize some profit
    - Minimize some cost
- **Optimization problems appear in so many applications**
  - ***Maximizing*** the number of jobs using a resource (Activity-Selection Problem)
  - Encode data in a file to ***minimize*** its size (Huffman Coding)
  - Collect the ***maximum*** value of goods that fit in a given bucket (Knapsack Problem)
  - Select the ***smallest*** weight of edges to connect all nodes in a graph (Minimum Spanning Tree)

Solving Optimization Problems

- There are two techniques for solving optimization problems

  1) Greedy Algorithms (Greedy Strategy)

  2) Dynamic Programming

- We still care about greedy algorithms because of some problems
  - Dynamic Programming is overkill (slow)
  - Greedy algorithm is simpler and more efficient

Greedy Algorithms

- A greedy algorithm is an approach for solving a problem by selecting the best option available at the moment. It doesn't worry whether the current best result will bring the overall optimal result.

- The algorithm never reverses the earlier decision even if the choice is wrong. It works in a top-down approach.

- This algorithm may not produce the best result for all the problems. It's because it always goes for the local best choice to produce the global best result.

Elements of Greedy Strategy

we can determine if the algorithm can be used with any problem if the problem has the following properties:

1. Greedy Choice Property

If an optimal solution to the problem can be found by choosing the best choice at each step without reconsidering the previous steps once chosen, the problem can be solved using a greedy approach. This property is called greedy choice property.

2. Optimal Substructure

If the optimal overall solution to the problem corresponds to the optimal solution to its subproblems, then the problem can be solved using a greedy approach. This property is called optimal substructure.

Activity Selection Problem

- The Activity Selection Problem is an optimization problem which deals with the selection of non-conflicting activities that needs to be executed by a single person or machine in a given time frame.

- Each activity is marked by a start and finish time. Greedy technique is used for finding the solution since this is an optimization problem.

What is Activity Selection Problem?

Let's consider that you have n activities with their start and finish times, the objective is to find solution set having **maximum number of non-conflicting activities** that can be executed in a single time frame, assuming that only one person or machine is available for execution.
Some **points to note** here:

- It might not be possible to complete all the activities, since their timings can collapse.

- Two activities, say **i** and **j**, are said to be non-conflicting if si >= fj or sj >= fi where si and sj denote the starting time of activities **i** and **j** respectively, and fi and fj refer to the finishing time of the activities **i** and **j** respectively.

- **Greedy approach** can be used to find the solution since we want to maximize the count of activities that can be executed. This approach will greedily choose an activity with earliest finish time at every step, thus yielding an optimal solution.

Activity Selection Problem

Input Data for the Algorithm:

- act[] array containing all the activities.
- s[] array containing the starting time of all the activities.
- f[] array containing the finishing time of all the activities.

Output Data from the algorithm:

sol[] array refering to the solution set containing the maximum number of non-conflicting activities

Steps for Activity Selection Problem

Following are the steps we will be following to solve the activity selection problem,

Step 1: Sort the given activities in ascending order according to their finishing time.

Step 2: Select the first activity from sorted array act[] and add it to sol[] array.

Step 3: Repeat steps 4 and 5 for the remaining activities in act[].

Step 4: If the start time of the currently selected activity is greater than or equal to the finish time of previously selected activity, then add it to the sol[] array.

Step 5: Select the next activity in act[] array.

Step 6: Print the sol[] array.

Example of Activity Selection Problem

Let's try to trace the steps of above algorithm using an example: In the table below, we have 6 activities with corresponding start and end time, the objective is to compute an execution schedule having maximum number of non-conflicting activities:

Start Time (s)	Finish Time (f)	Activity Name
5	9	a1
1	2	a2
3	4	a3
0	6	a4
5	7	a5
8	9	a6

A possible **solution** would be:

Example of Activity Selection Problem

Step 1: Sort the given activities in ascending order according to their finishing time.
The table after we have sorted it:

Start Time (s)	Finish Time (f)	Activity Name
1	2	a2
3	4	a3
0	6	a4
5	7	a5
5	9	a1
8	9	a6

Step 2: Select the first activity from sorted array act[] and add it to the sol[] array, thus **sol = {a2}**.

Example of Activity Selection Problem

Step 3: Repeat the steps 4 and 5 for the remaining activities in act[].

Step 4: If the start time of the currently selected activity is greater than or equal to the finish time of the previously selected activity, then add it to sol[].

Step 5: Select the next activity in act[]

Example of Activity Selection Problem

For the data given in the above table,

A.Select activity **a3**. Since the start time of **a3** is greater than the finish time of **a2** (i.e. s(a3) > f(a2)), we add **a3** to the solution set. Thus **sol = {a2, a3}**.

B.Select **a4**. Since s(a4) < f(a3), it is not added to the solution set.

C.Select **a5**. Since s(a5) > f(a3), **a5** gets added to solution set. Thus **sol = {a2, a3, a5}**

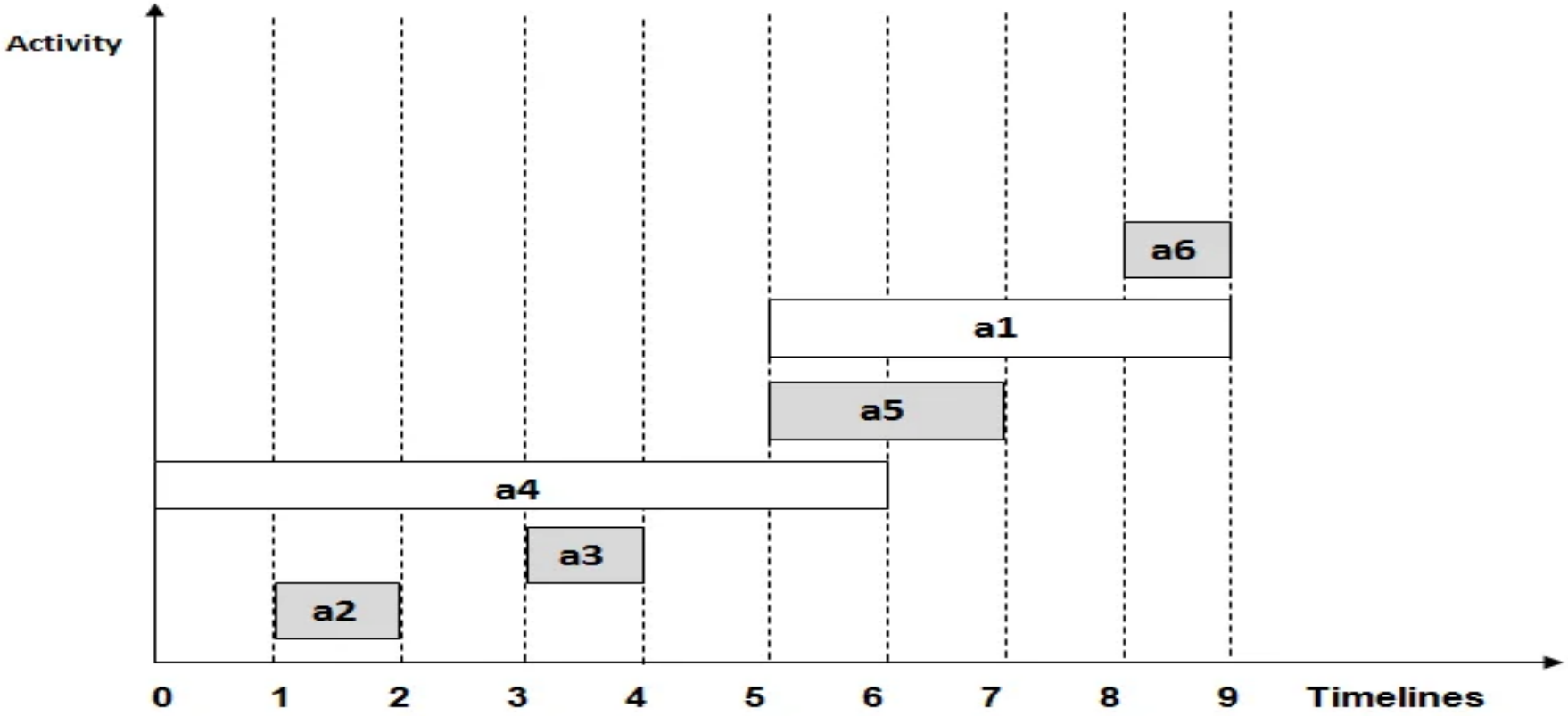
D.Select **a1**. Since s(a1) < f(a5), **a1** is not added to the solution set.

E.Select **a6**. **a6** is added to the solution set since s(a6) > f(a5). Thus **sol = {a2, a3, a5, a6}**.

Step 6: At last, print the array sol[]

Example of Activity Selection Problem

Hence, the execution schedule of maximum number of non-conflicting activities will be



In the above diagram, the selected activities have been highlighted in grey

Real-life Applications of Activity Selection Problem

Following are some of the real-life applications of this problem:

- Scheduling multiple competing events in a room, such that each event has its own start and end time.

- Scheduling manufacturing of multiple products on the same machine, such that each product has its own production timelines.

- Activity Selection is one of the most well-known generic problems used in Operations Research for dealing with real-life business problems.